



Introduction and Overview

Rizoan Toufiq

Assistant Professor

Department of Computer Science & Engineering

Rajshahi University of Engineering & Technology

Rajshahi-6204

Outline

-
- Basic Terminology
 - Data Structure
 - Classification of Data Structure
 - Data Structure Operations
 - Abstract Data Type (ADT)
 - Algorithm

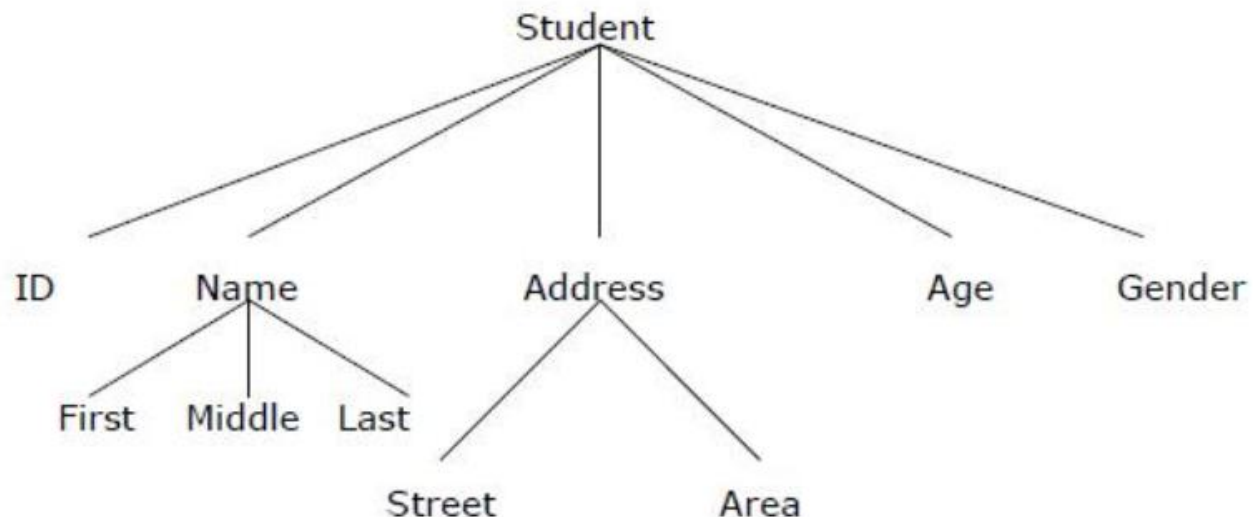
Basic Terminology

Basic Terminology

- **Data** are values or set of values.
- **Data item** refers to a single unit of values.
- Data items, that are divided into sub items, are **group items**;
- Data items that are not divided into sub items are called **elementary items**.

Basic Terminology

- (ID, Age, Gender, First, Middle, Last, Street, Area) are **elementary data items**
- (Name, Address) are **group data items**



Basic Terminology

- An **entity** is something that has certain **attributes** or **properties** which may be assigned some **values**.
- The **values** themselves may be either **numeric** or **non-numeric**

Attributes:	Name	Age	Gender	Social Society number
Values:	Hamza	20	M	134-24-5533
	Ali Rizwan	23	M	234-9988775
	Fatima	20	F	345-7766443

Basic Terminology

- An **entity set** is a **group** of or set of **similar entities**.
- For example,
 - employees of an organization,
 - students of a class etc.
- Each **attribute** of an entity set has a **range of values**.
- **Information** is sometimes used for
 - data with given attributes,
 - in other words meaningful or processed data

Basic Terminology

- A **field** - an **attribute** of an entity
- A **record** - the **collection** of **field values** of a given entity
- A **file** - the **collection** of **records** of the entities in a given entity set.
- A **key** - take(s) **unique values** and can be used to distinguish one record from the others.

Attributes:	Name	Age	Gender	Social Society number
Values:	Hamza	20	M	134-24-5533
	Ali Rizwan	23	M	234-9988775
	Fatima	20	F	345-7766443

Basic Terminology

- **Records** may also be classified to **length**.
 - **Fixed-length** records – contain same data items with the same amount of space assigned to each data item.
 - **Variable-length** record – contain different length i.e. **student record** since different student take different number of courses.

Data Structure

Data Structure

- Data may be organized in many different ways; the **logical** or **mathematical model** of a particular organization of data is called a **data structure**.

Classification of Data Structure

Classification of Data Structure

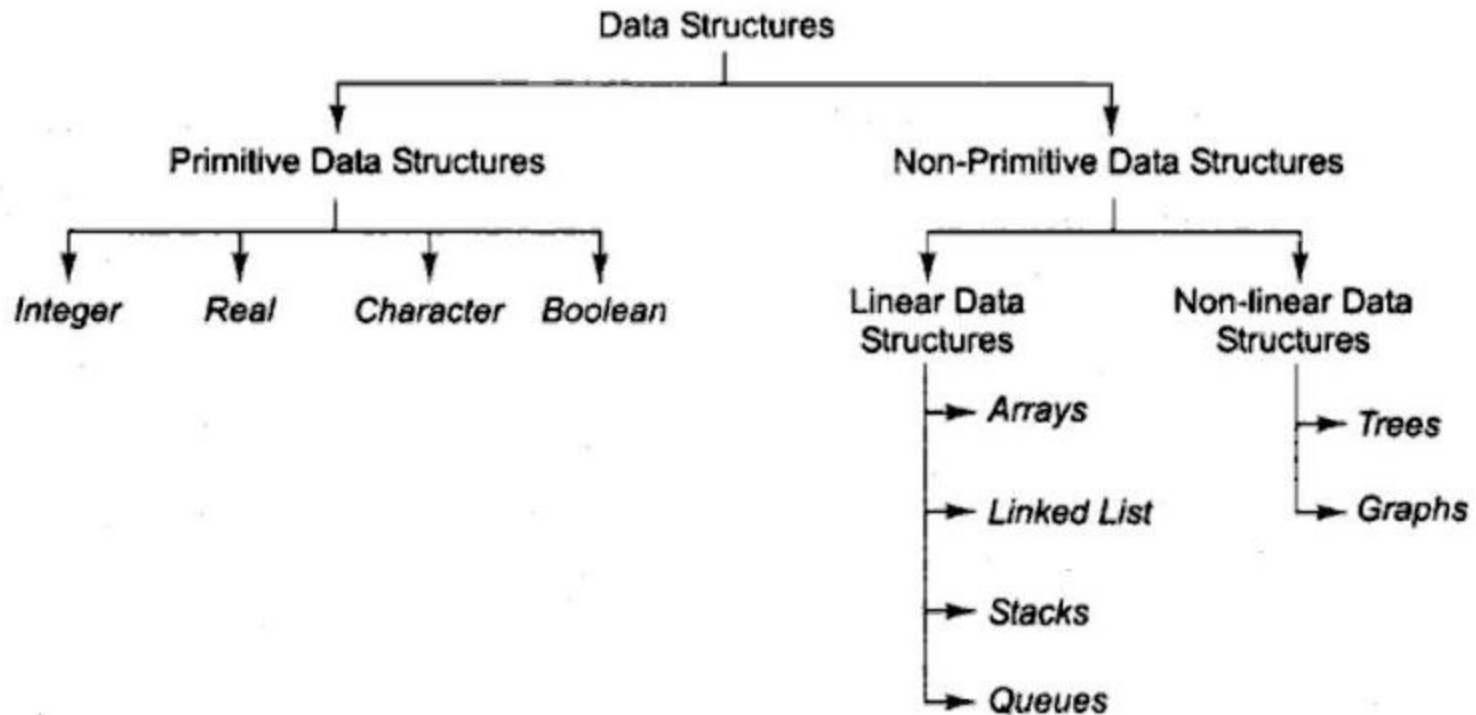


Fig. 1.1 Classification of Data Structures

Classification of Data Structure (Array)

- Linear data structure
- Suppose A is an array
 - **Subscript Notation:**

$$A_1, A_2, A_3, \dots, A_n$$

- **Parenthesis Notation**

$$A(1), A(2), A(3), \dots, A(n)$$

- **Bracket Notation**

$$A[1], A[2], A[3], \dots, A[n]$$

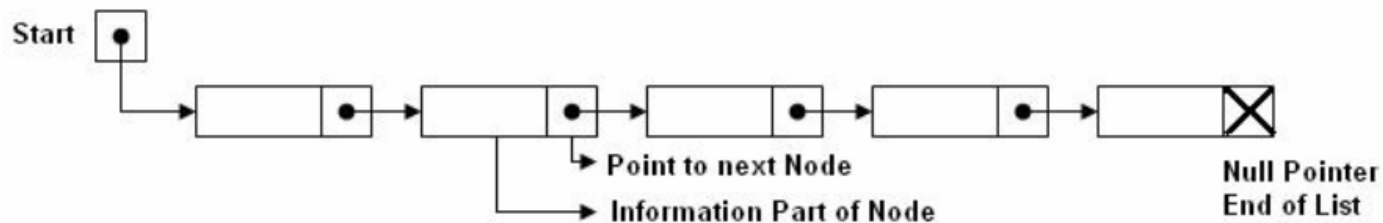
1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

A[0] A[1] A[2] A[3] A[4] A[5] A[6] A[7]

```
int A[8] = {1, 2, 3, 4, 5, 6, 7, 8};
```

Classification of Data Structure (Linked List)

- A linked list or one way list is a **linear** collection of data elements, called nodes, where the linear order is given by means of pointers.
- Each node is divided into **two** parts:
 - The first part - the **information** of the element/node
 - The second part- the address of the next node
- There is a special pointer **Start/List** contains the address of **first** node in the list.
- If this special **pointer** contains **null**, means that **List** is **empty**.



Classification of Data Structure (Linked List)

	Customer	Salesperson
1	Adams	Smith
2	Brown	Ray
3	Clark	Jones
4	Drew	Ray
5	Evans	Smith
6	Farmer	Jones
7	Geller	Ray
8	Hill	Smith
9	Infeld	Ray

Fig. 1.4

Classification of Data Structure (Linked List)

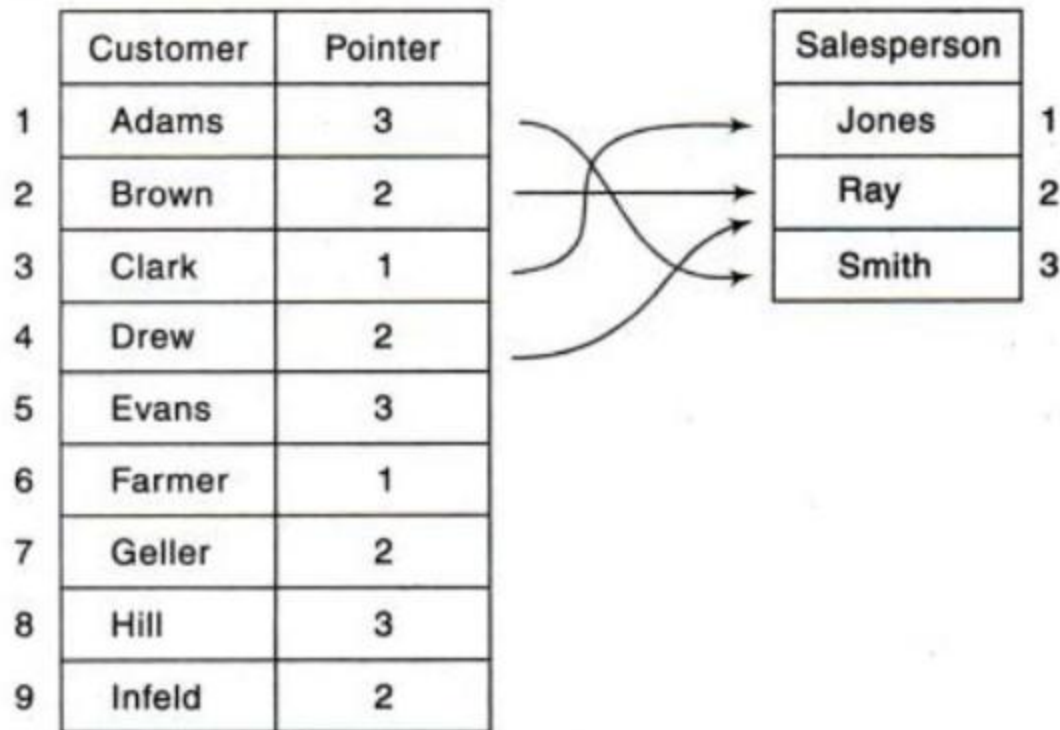


Fig. 1.5

Classification of Data Structure (Linked List)

	Salesperson	Pointer
1	Jones	3, 6
2	Ray	2, 4, 7, 9
3	Smith	1, 5, 8

Fig. 1.6

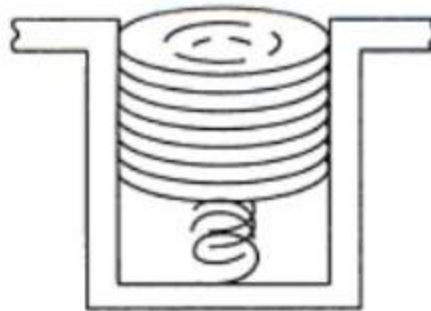
	Customer	Link
1	Adams	5
2	Brown	4
3	Clark	6
4	Drew	7
5	Evans	8
6	Farmer	0
7	Geller	9
8	Hill	0
9	Infeld	0

Fig. 1.7

Salesperson	Pointer	
Jones	3	1
Ray	2	2
Smith	1	3

Classification of Data Structure (Stack)

- List-in-First-out (**LIFO**)
- A linear list in which insertion and deletions can take place only at one end, called the **top**.



(a) Stack of dishes

Classification of Data Structure (Queue)

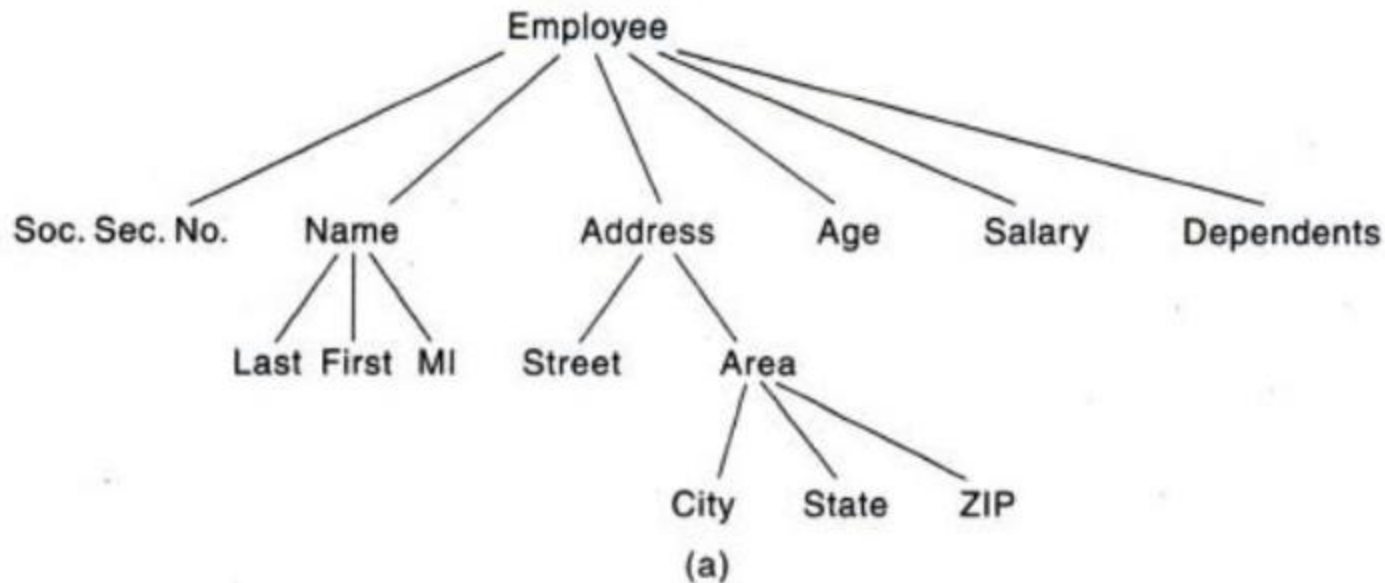
- First-in-First-out (**FIFO**)
- A linear list in which
 - deletions can take place only at one end of the list (The **front** of the list)
 - Insertion can take place only at the other end of the list (The **rear** of the list)



(b) Queue waiting for a bus

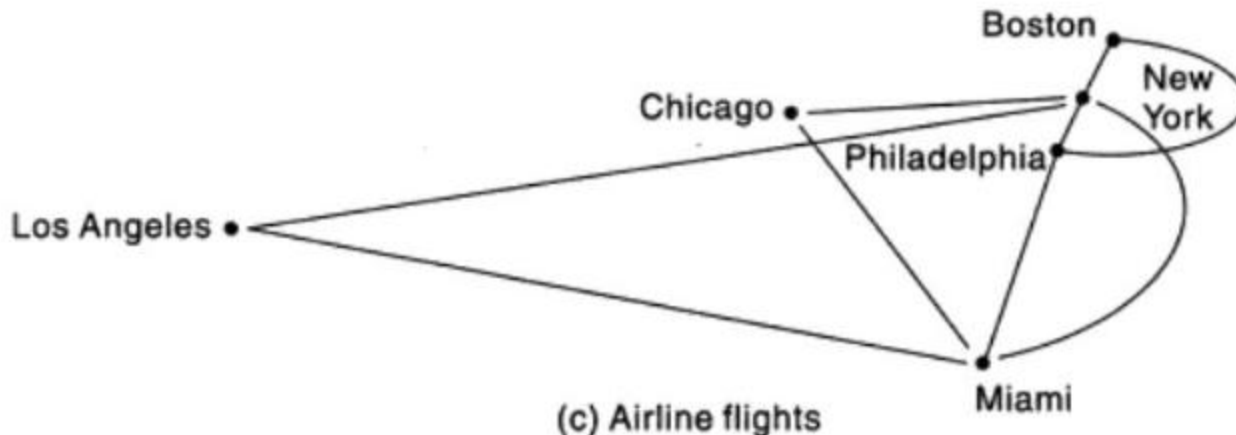
Classification of Data Structure (Tree)

- Data frequently contain a **hierarchical** relationship between various elements.
- The data structure which reflects this relationship is called a **rooted** tree or simply a tree.



Classification of Data Structure (Graph)

- Data sometimes contains a relationship between pairs of elements which is **not necessarily hierarchical in nature**, e.g. an **airline flights** only between the cities connected by lines.
- This data structure is called **Graph**.



Data Structures Operations

Data Structures Operations

- **Traversing:** accessing each record/node exactly once so that certain items in the record may be processed. (This accessing and processing is sometimes called “visiting” the record.)
- **Searching:** Finding the location of the desired node with a given key value, or finding the locations of all such nodes which satisfy one or more conditions.
- **Inserting:** Adding a new node/record to the structure.
- **Deleting:** Removing a node/record from the structure.

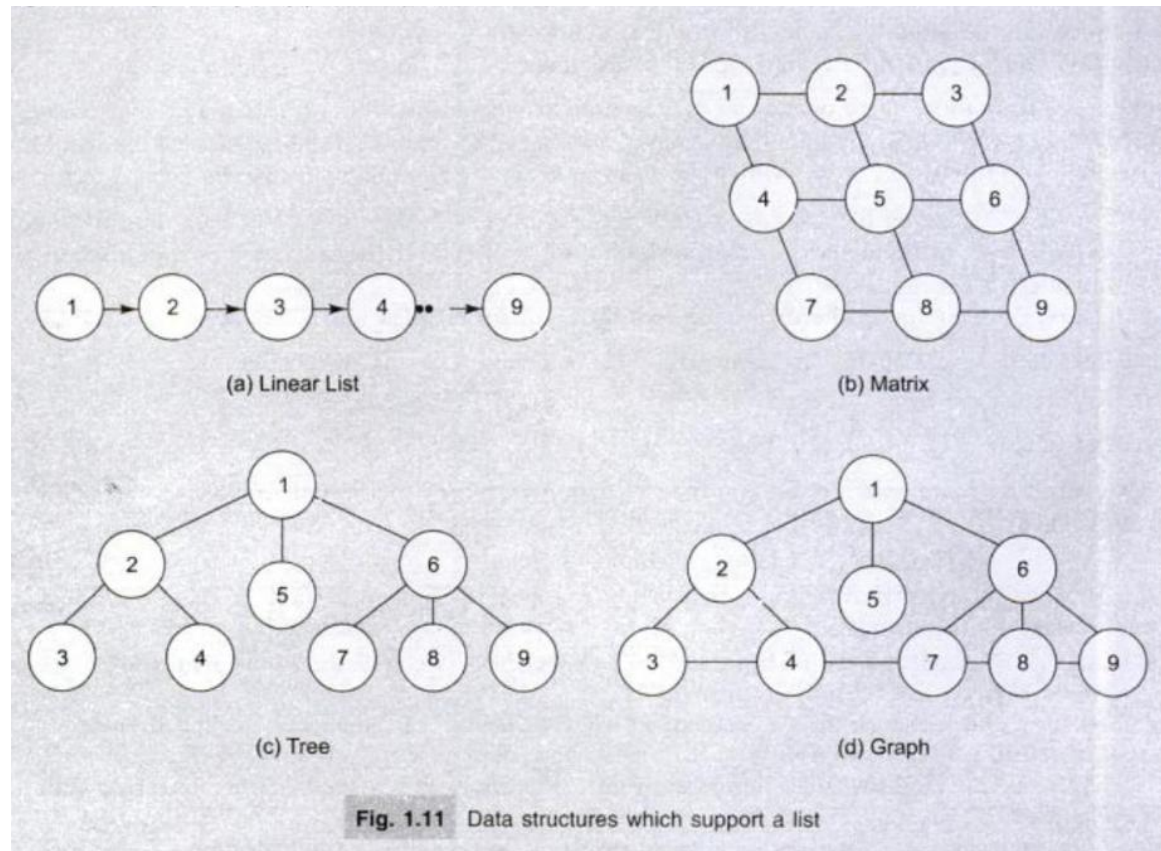
Abstract Data Type(ADT)

Abstract Data Type(ADT)

- An abstract data type (**ADT**) refers to a set of data values and associated operations that are specified accurately, independent of any particular implementation.
- In broader terms, the ADT consists of a **set of definitions** that allow us to use the functions while hiding the implementation.
- Let a list L consisting of data items – 1, 2, 3, 4, 5, 6, 7, 8, 9

Abstract Data Type(ADT)

- Let a list L consisting of data items – 1, 2, 3, 4, 5, 6, 7, 8, 9



Abstract Data Type(ADT)

- Assume that we place the list on an **ADT**.
- The **user should not** aware of the structure that we use. i.e. whether it is a tree, graph or something else.
- As long as they are able to **insert and retrieve data**.
- It does not make a difference as to how we store the data.

Abstract Data Type(ADT)

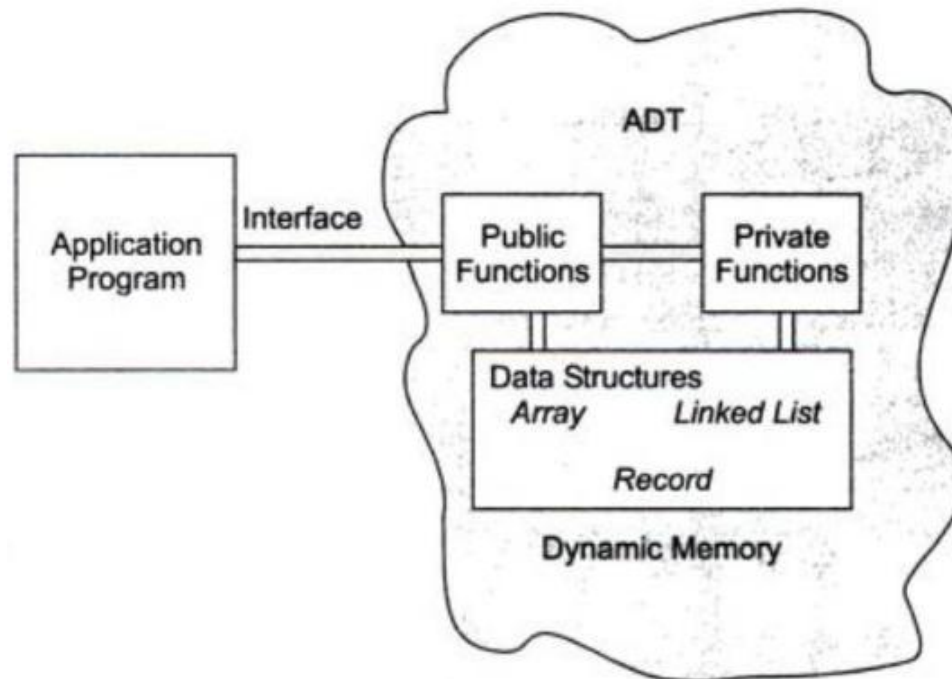


Fig. 1.13 ADT Model

Abstract Data Type(ADT)

- Assume that we place the list on an **ADT**.
- The **user should not** aware of the structure that we use. i.e. whether it is a tree, graph or something else.
- As long as they are able to **insert and retrieve data**.
- It does not make a difference as to how we store the data.

Algorithm

Algorithm

- **Definition:**
 - A well-defined list of steps for solving a particular problem.
- **Objective:**
 - Develop efficient algorithm for the processing of our data.
- Two major **measures** of the efficiency of an algorithm are:
 - Time and
 - Space
- The **complexity of an algorithm** is the **function** which gives the running time and/or space in term of the input size. [More details in **chapter 2**]

Algorithm

- The choice of data structure involves a **time-space tradeoff**.
- **Time-Space Tradeoff:**
 - One algorithm may require less memory space but may take more time to complete its execution.
 - On the other hand, the other algorithm may require more memory space but may take less time to run to completion.
 - Thus, we have to sacrifice one at the cost of other.
 - In other words, there is **Space-Time trade-off** between algorithms.

Algorithm

- **Searching** algorithm:
 - **Linear Search**
 - The average number of comparisons for a file with n records is equal to $n/2$ i.e. the complexity of linear search is $C(n) = n/2$.
 - **Binary Search**
 - The complexity of binary search algorithm is $C(n) = \log_2 n$
 - List must be sorted.

- Suppose a file of records contains **names, social security number** and much additional **information** among its field.

Rizoan Toufiq, Assistant Prof, Dept. of CSE, RUET

Algorithm

(An Example of time-space tradeoff)

- **Sorting** the file alphabetically and running **binary search** is a very efficient way to find the record for a given name.

Algorithm

(An Example of time-space tradeoff)

37

- **Problem definition:**
 - Suppose we are given only the **social security number** of some persons and **name** of other person.



Algorithm

(An Example of time-space tradeoff)

- **Problem definition:**
 - Suppose we are given only the **social security number** of some persons and **name** of other person.
- **First Solution:**
 - Apply linear search for the record, which is extremely **time-consuming** for a very large number of record.

Algorithm

(An Example of time-space tradeoff)

- **Problem definition:**
 - Suppose we are given only the **social security number** of some persons and **name** of other person.
- **Second Solution:**
 - Create another file which is sorted numerically according to social security number.
 - This would double the space required for sorting the data

Algorithm

(An Example of time-space tradeoff)

- **Problem definition:**
 - Suppose we are given only the **social security number** of some persons and **name** of other person.
- **Third Solution:**
 - **Sort** the **main file** numerically by **social security number**
 - Have a **auxiliary array** with only two columns,
 - the **first column** containing an alphabetized list of the name and
 - the **second column** containing pointer which give the locations of the corresponding records in the main file
 - Required additional space.

Algorithm

(An Example of time-space tradeoff)

- **Problem definition:**
 - Suppose we are given only the **social security number** of some persons and **name** of other person.

- **Third Solution:**

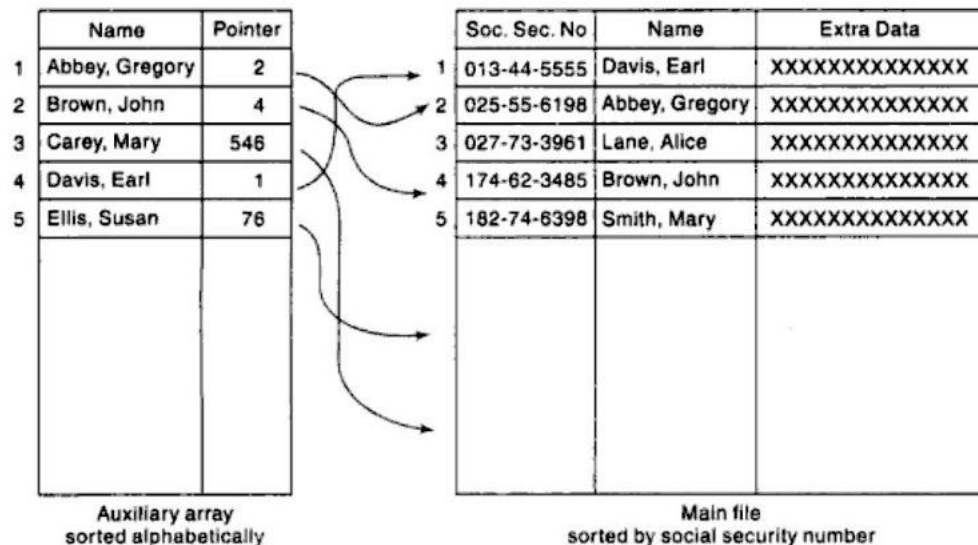


Fig. 1.14

END...